

Cloud Run Plugin for PipeCD v1

Google Summer of Code 2026 Proposal

Author: Dawid Wnętrzak

Contents

Cloud Run Plugin for PipeCD v1.....	1
Google Summer of Code 2026 Proposal.....	1
Author: Dawid Wnętrzak.....	1
OVERVIEW.....	2
ARCHITECTURE.....	2
Cloud Run → PipeCD mapping.....	3
Plugin Responsibilities.....	3
Deployment Flow (MVP-focused).....	4
Extensibility.....	5
EDGE CASES.....	5
Implementation Plan.....	6
Milestones.....	7
Project Timeline.....	8
Future Work.....	8

OVERVIEW

PipeCD v0 tightly couples platform-specific deployment logic with the core system, making it difficult to maintain and extend.

PipeCD v1 introduces a plugin-based architecture that separates concerns:

- the core handles orchestration, state management, and deployment strategies
- plugins handle platform-specific execution

This project aims to implement a Cloud Run plugin that aligns with this architecture and enables clean, extensible support for Cloud Run deployments.

ARCHITECTURE

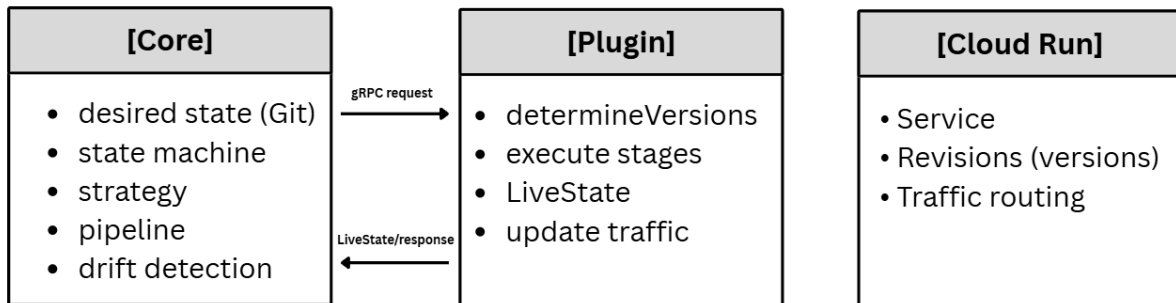


Figure 1: PipeCD v1 architecture – separation between core orchestration and platform-specific plugin execution.

PipeCD v1 follows a plugin-based architecture where responsibilities are clearly separated.

The core acts as the orchestrator, managing the deployment state machine, strategy execution, drift detection, and pipeline control.

The plugin implements the gRPC contract and performs platform-specific operations such as deploying revisions, updating traffic, and checking readiness.

This separation allows the core to remain platform-agnostic while plugins encapsulate all platform-specific behaviour.

Cloud Run → PipeCD mapping

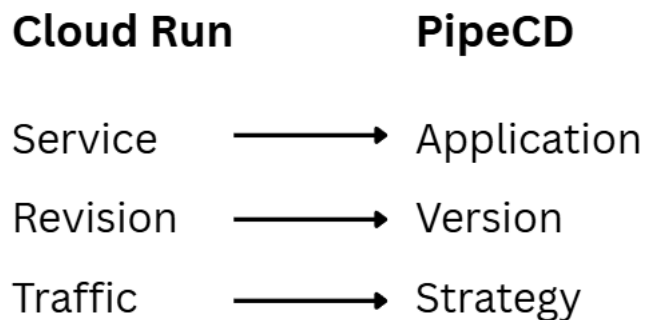


Figure 2 Mapping between Cloud Run resources and PipeCD concepts.

The Cloud Run resource model maps directly to PipeCD concepts.

A Cloud Run Service corresponds to a PipeCD application, representing the logical deployment unit.

Each Cloud Run Revision is an immutable deployment and maps to a PipeCD version.

Traffic distribution across revisions is controlled by PipeCD's deployment strategy, with the plugin applying traffic changes on the platform.

Plugin Responsibilities

The Cloud Run plugin is responsible for executing all platform-specific operations while remaining stateless and driven by the core.

Its responsibilities include:

- determining deployment versions (current, target, stable)
- providing platform-specific pipeline stages based on the selected deployment strategy
- executing platform operations within each stage (e.g. revision creation, traffic updates, readiness checks)
- providing current platform state via the LiveState service
- optionally supporting deployment preview and validation

The plugin does not make deployment decisions. It provides platform-specific behaviour and executes operations, while the core remains responsible for orchestrating the deployment flow, managing the state machine, and controlling the pipeline execution.

Deployment Flow (MVP-focused)

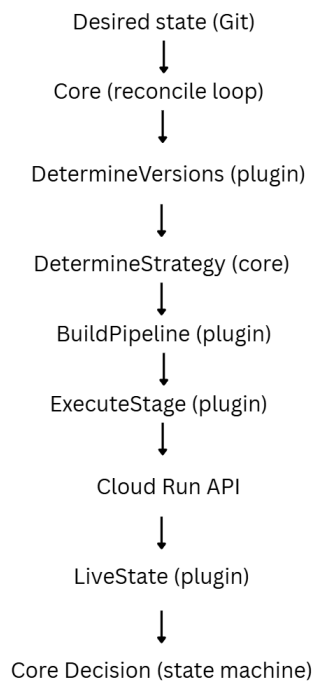


Figure 3 : Deployment flow where the core reconciles the desired state from Git and orchestrates the execution of the deployment pipeline.

Direct rollout (MVP)

1. create revision (v2)
2. wait until ready
3. switch traffic 100% to v2

Figure 4 MVP deployment flow using direct strategy (full traffic switch).

In PipeCD v1, deployments are performed by building a deterministic pipeline

based on the desired state stored in Git.

The core reconciliation loop compares the desired and actual state, selects the deployment strategy, and requests the plugin to provide the platform-specific pipeline stages required for execution.

The plugin is responsible only for platform-specific logic, such as determining deployment versions, providing pipeline stage definitions, executing operations on the Cloud Run API within each stage, and reporting the current state via the LiveState service.

The core controls the state machine, orchestrates the execution of the pipeline, and decides whether the deployment should continue, retry, or fail.

The generated pipeline is deterministic and idempotent, ensuring consistent reconciliation behaviour even in the presence of retries, partial failures, or external changes.

Extensibility

The architecture is designed to support more advanced deployment strategies in future iterations.

While the MVP focuses on a direct deployment strategy, the design already aligns with PipeCD's strategy-driven pipeline model. This allows seamless extension to support canary or progressive delivery without requiring changes to the core orchestration logic, as new strategies can reuse the same pipeline-driven execution model.

Similarly, the initial implementation of DetermineVersions may focus on current and target revisions, with support for tracking the last stable revision added in later stages.

EDGE CASES

The following edge cases are considered to ensure reliable behavior of the MVP implementation:

- Drift due to manual changes

If the actual state is modified outside of PipeCD (e.g. manual traffic update), the core detects the drift via LiveState and reconciles it back to the desired state.

- Revision readiness timeout

If a newly created revision does not become ready within a defined time, the deployment fails and triggers retry or termination logic.

- Invalid desired state or deployment failure

If the desired state results in an unrecoverable error (e.g. invalid configuration), the pipeline fails safely, allowing correction and redeployment.

- Idempotent stage execution

Each stage must be safe to retry without causing unintended side effects on Cloud Run resources. This ensures reliability in case of transient failures or repeated reconciliation loops.

Implementation Plan

The implementation will follow a step-by-step approach, starting from a minimal working plugin and gradually adding functionality required for a complete Cloud Run deployment flow.

The focus is to deliver a working MVP early (direct deployment), and then improve reliability, integration, and developer experience.

Phase 1 - Plugin Setup

At the beginning, I will set up a basic plugin that can communicate with PipeCD core.

This includes implementing the gRPC server, integrating with the existing plugin SDK, and making sure all required services (DeploymentService, LivestateService, PlanPreviewService) are exposed.

At this stage, the goal is not full functionality, but a working communication flow between core and plugin.

Phase 2 - LiveState (reading actual state)

Next, I will focus on reading the current state from Cloud Run.

This includes calling the Cloud Run API, mapping services, revisions, and traffic into PipeCD concepts, and exposing this through the LiveState service.

This is important because the core relies on this information to detect drift and decide what to do next.

Phase 3 - Deployment (MVP)

Once the plugin can read the state, I will implement the actual deployment logic using a direct strategy.

This means:

- creating a new revision
- waiting until it becomes ready
- shifting 100% of traffic to the new revision

The plugin will provide the required stages, while the core will orchestrate their execution.

At this point, the full deployment loop (desired → actual) should already work end-to-end.

Phase 4 - Reliability

After the MVP is working, I will focus on making it stable and safe.

This includes handling retries, making sure all operations are idempotent, and properly dealing with failures such as timeouts or invalid configurations.

The goal is to ensure that repeated executions of the same pipeline do not break the system.

Phase 5 - SDK Improvements

While implementing the plugin, I will also review the current plugin SDK.

If I identify missing abstractions or limitations, I will propose and implement improvements where appropriate.

This will help future contributors build plugins more easily.

Phase 6 - Documentation and Blog

Finally, I will prepare documentation describing how to build a PipeCD plugin and how to use the Cloud Run plugin.

I will also write a blog post summarizing the approach, design decisions, and lessons learned during the project.

Milestones

The project will be delivered in incremental steps, ensuring that a working deployment flow is achieved early, followed by stability improvements and documentation.

Weeks 1-2: Plugin setup and integration with PipeCD core (gRPC services, basic communication)

Week 3: LiveState implementation (reading and mapping Cloud Run state)

Weeks 4-5: Deployment implementation (direct strategy: revision creation, readiness check, traffic switch)

Milestone: end-to-end deployment working

Weeks 6-7: Reliability improvements (idempotency, retries, failure handling)

Week 8: SDK improvements (if required based on implementation)

Weeks 9-10: Documentation and blog post, final polish

Project Timeline

PROJECT GANTT CHART

	TASK	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10
FOUNDATION	PLUGIN SETUP & GRPC										
	CORE INTEGRATION										
STATE	CLOUD RUN API & MAPPING										
	LIVESTATE										
DEPLOYMENT	PIPELINE STAGES										
	DEPLOYMENT (MVP)					MVP DELIVERED					
RELIABILITY	RELIABILITY, TESTING & EDGE CASES										
FINALIZATION	DOCS & BLOG										

Figure 5: High-level project timeline showing the implementation of the Cloud Run plugin. A working deployment flow (MVP) is delivered by Week 5, followed by reliability improvements, testing, and final documentation.

Future Work

The initial implementation focuses on a direct deployment strategy and core functionality required for a working Cloud Run plugin.

Future extensions may include support for more advanced scenarios, such as:

- handling post-deployment failures after traffic shift, including rollback to the last stable revision
- supporting progressive delivery strategies (e.g. canary) with partial traffic control
- tracking and maintaining stable revisions for safer recovery and rollback operations

These enhancements can be built on top of the existing pipeline-based architecture without requiring changes to the core system.